cuButterfly: Unleashing Performance of Butterfly Computation with Hierarchical Hybrid Dataflow

ISCA 2026 Internal Working Draft – Anonymous – Do Not Distribute

***Abstract*—Butterfly computation underlies Fourier, number-theoretic, Hadamard, and zeta transforms. Their shared dependencies expose a common performance problem. Spatial expansion consumes execution resources and live storage, while temporal reuse requires state retention or repeated communication. Their balance depends on the arithmetic, workload, and hardware. We characterize these demands jointly so that organization can be optimized under a target's capacity, bandwidth, and execution constraints.**

We present CUBUTTERFLY, which derives dependency-closed local regions, their communication and readiness relations, and resource demands from butterfly dependencies. *Hierarchical hybrid dataflow* combines spatial expansion and temporal reuse along stage and data dimensions, repeating the construction across storage scopes. The resulting organization method reuses these relations across arithmetic cores and workloads while evaluating demand against target hardware constraints.

An A100 80GB evaluation observes geometric-mean latency ratios of 1.298× and 1.366× over cuFFT for its FP32 and FP64 FFT cohorts, including multidimensional requests. An independent cyclic NTT study reaches 1.308× over GPU-NTT across 54 matched 32/64-bit-word requests. Separate fixed-core controls improve complete-call performance by approximately 1.22× with either FFT leaf, with local organization gains also observed for FWHT and NTT. These measurements support reusable organization on the tested workloads. Selected CKKS complete operations improve by 1.131× over the faster of two native execution choices across 36 short-chain requests. Fixed-core controls distinguish organization gains from core substitution, while FP16 and application losses expose limits. Floating-point errors are reported without a fixed acceptance threshold.

I. INTRODUCTION

Butterfly computation is a common building block in scientific computing, machine learning, and cryptography. Fourier transforms support fast convolution and long-sequence models [1]–[3]. Number-theoretic transforms provide exact polynomial arithmetic [4], [5], while Hadamard transforms support low-bit model quantization [6]. Learned butterfly factorizations also expose structured linear layers [7]. These computations have different numerical contracts but repeatedly combine pairs of values along logarithmic stage sequences. This recurring structure raises a performance question: which execution principles carry across their arithmetic and applications?

Mature GPU libraries already optimize both arithmetic and data movement [8]–[11]. Transform generators such as SPIRAL and FFTX also reuse mathematical descriptions to generate implementations [12], [13]. The same dependency pattern is repeatedly optimized within separate domains, yet its performance changes with the surrounding workload and target. Arithmetic-core quality alone does not explain those changes.

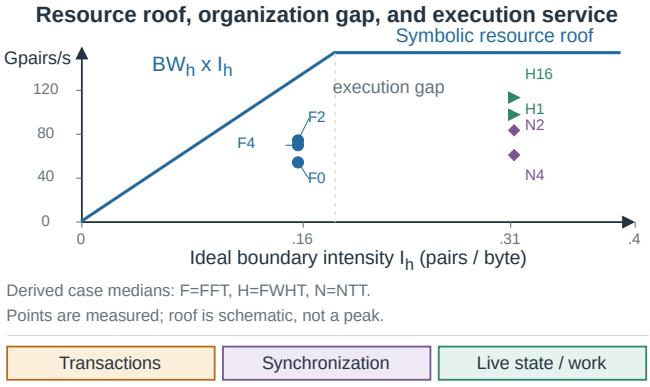


Fig. 1. Measured FFT, FWHT, and NTT organizations attain different rates at equal ideal intensity within each family. The roof is symbolic, not a measured hardware peak.

Arithmetic intensity and Roofline [14] provide a starting point for understanding this variation. Yet matched organizations with the same arithmetic and equal ideal data-reuse intensity attain different pair-update rates on the same device (Figure 1). Ideal reuse alone does not explain the result, because organization also changes actual transactions and resource use. This motivates examining **execution organization**: which operations run together, where intermediate values live, and how they reach their consumers. Retaining more values can raise an ideal bandwidth ceiling, but it also consumes local capacity, communication service, and concurrency. Even abundant parallel work can therefore use the hardware inefficiently.

The effect is measurable independently of core replacement. FFT controls improve complete-call performance by about 1.22× while holding each of two arithmetic leaves fixed (Section VI-C). Further FFT, FWHT, and NTT controls show that deeper retention can help or hurt (Section II-B). A single preferred grouping is thus insufficient. Because the pairing dependencies recur across these families, the question is **whether a common organization method can jointly manage local reuse, communication, and parallel execution across different arithmetic cores, workloads, and hardware.**

CUBUTTERFLY addresses this question with **hierarchical hybrid dataflow**. Its central construction derives **dependency-closed local regions**, communication and readiness relations between them, and their resource demands. Spatial and temporal choices along the stage and data dimensions then determine live storage, communication, execution demand, and available work. The target supplies capacities and service rates against

which these demands are evaluated. Repeating the construction across storage scopes handles workloads beyond one local capacity. At each boundary, the producer can write values in a representation suitable for its successor. This must reconcile changing access groups with limited storage ports, so local reuse does not merely replace transfers with bank conflicts.

Arithmetic, workload, and hardware become separate inputs whose performance effects are evaluated jointly. Generation constructs compatible realizations, and measurement confirms choices within the executable space. Search instantiates this method; it is not a separate contribution or a guarantee of optimal mapping.

Our contributions are three linked pieces:

1) We derive local regions and their communication and readiness relations from butterfly dependencies, connecting space–time organization to storage, execution demand, and available parallel work.
2) We develop **hierarchical hybrid dataflow** to optimize these choices under hardware constraints, using dependency-closed regions and explicit handoffs while retaining specialized arithmetic cores.
3) We realize the construction through subgraph generation and evaluate library performance, organization effects, and the resource behavior of generated implementations.

The A100 80GB operator evaluation reports FFT geometric-mean speedups of 1.298× and 1.366× over cuFFT for its FP32 and FP64 cohorts, including multidimensional requests, with an independent cyclic NTT study reaching 1.308× over GPU-NTT and FP32 FWHT reaching 1.474× over DaoFHT. Section VI distinguishes external comparisons, fixed-core controls, and complete applications, including their remaining gaps.

II. Background and Motivation

The organization question begins with the dependency graph. We first identify its recurring structure and numerical variants, then use measured and analytical examples to derive the requirements for a common method.

A. Butterfly computation and its variants

A butterfly computation combines small groups of values through a sequence of stages. In a radix-two stage, disjoint pairs can execute independently, but the next stage pairs their outputs differently. These changing partners are what make local computation easy to parallelize and the complete computation difficult to organize. For $N = 2^L$, the graph has L stages and $N/2$ pair updates per stage. A stage associated with bit q pairs indices i and $i \oplus 2^q$. The stage sequence defines the dependency envelope, while the pair function supplies arithmetic and coefficients. Batch instances add independent copies of the graph. **Shared structure does not imply interchangeable semantics.**

The family includes complex FFT, finite-field NTT, sum-and-difference FWHT, subset and superset zeta transforms, and products of structured 2×2 stages. One-sided zeta updates use a subset of the full pairing dependencies. For example,

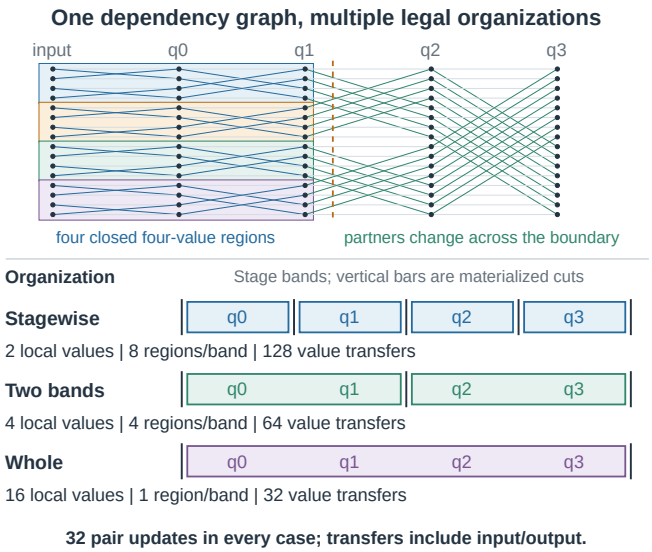


Fig. 2. Grouping stages trades local state and independent regions for fewer transfers in the same Hadamard graph. Counts are analytical.

an FFT pair multiplies by a complex twiddle, an NTT pair performs modular arithmetic, and an FWHT pair uses only sums and differences. A common organization must retain these distinctions rather than replace every pair with one arithmetic implementation. Multidimensional separable transforms compose axis transforms with a shared tensor layout, making inter-axis placement part of the same execution problem.

Cooley–Tukey (CT) and Gentleman–Sande (GS) illustrate another distinction [1], [15]. Their local forms place twiddle multiplication on opposite sides of addition and subtraction. Forward and inverse computation also change coefficients, stage order, permutations, and normalization. Stockham algorithms distribute permutations across passes [16]. Section III-D explains which structural relations survive these changes.

The requested precision, direction, normalization, and input/output layout constrain implementation choices. Internal reordering is legal only when it preserves the request, with adaptation included in execution cost.

The same graph admits different organizations. Figure 2 holds the 16-point Hadamard graph fixed. Stagewise execution exposes eight independent pairs and transfers all values at every stage. Two-stage groups retain four values each and halve materialized traffic. One whole-transform group minimizes traffic but exposes one independent region per transform. These region counts do not fix worker counts, since workers can cooperate within a region.

Storage scopes give these logical choices physical costs [17]. Worker-local storage, shared storage, and external storage offer different capacities and exchange rates. A generated codelet or imported core such as cuFFTDx [18] must operate within those limits. The preferred grouping therefore depends on both arithmetic service and the surrounding organization.

B. From a resource roof to an execution organization

Changing pair partners makes data retention and parallel execution interdependent. Roofline bounds their performance opportunity but does not map stages, data, and storage together. We examine the same arithmetic under different organizations to derive that mapping.

Reuse raises a resource ceiling. Figure 1 contrasts a symbolic resource roof with measured execution for FFT, FWHT, and NTT. The Roofline opportunity [14] comes from retaining values between dependent stages. For B transforms of length $N = 2^L$, let e be bytes per value and G the number of materialized stage groups. There are $BNL/2$ pair updates. If each group reads and writes the array once, ideal value-only intensity is

$$I_{\text{value}} \simeq \frac{L}{4eG}. \tag{1}$$

At a selected storage boundary, actual intensity is pair work divided by transferred bytes. Its roofline ceiling is the smaller of core throughput and bandwidth times intensity. Coefficients, transactions, cache reuse, and spills change the traffic, while synchronization and execution efficiency determine the distance below this ceiling.

Reducing G raises ideal intensity while changing live state, ownership, communication, and synchronization. The trade-off in Figure 2 therefore affects both the resource ceiling and the execution that approaches it. **The research question is how a parallel organization realizes the available hardware performance.**

Limiting cases expose different organization choices. The H1/H16 points use FWHT FP64 at $N = 2^{20}$ and $B = 1/16$ with two materialized bands. Sixteen times as much independent work raises pair rate from 97.8 to 113.4 Gpairs/s at unchanged ideal intensity. Batch can amortize startup and fill idle slots, but cannot by itself resolve an inefficient organization. These timings show limited scaling; traffic and resource measurements are needed to establish its cause.

At $N = 2^{20}, B = 16$, F0/F2/F4 use FFT FP64 with local groups of 0/2/4 stages, and N2/N4 use word64 NTT with groups of 2/4. All retain two materialized bands. Pair rate is $BNL/(2T)$, and ideal intensity follows Eq. 1. FFT reaches 54.4, 74.4, and 70.0 Gpairs/s with the same arithmetic and outer boundary. Local reuse helps, but deeper retention does not continue the gain. NTT instead falls from 83.6 to 61.1 Gpairs/s. Thus equal ideal intensity permits different execution rates, and the preferred local organization depends on arithmetic service and resource use.

Two analytical limits extend this reasoning. If a target doubles the applicable resource roof but a serialized exchange keeps throughput unchanged, efficiency relative to that roof halves. Doubling only arithmetic capacity need not raise a bandwidth-limited roof, so the applicable bound matters. Separately, a larger transform can outgrow local storage despite abundant parallel work, requiring smaller resident tile subgraphs and larger-scope exchanges. These are analytical hardware and capacity cases, not measured cross-GPU efficiency results.

The trade-off recurs across butterfly families. The full controls in Figure 12 retain each request's scalar arithmetic primitive and external storage boundaries. Two-stage local grouping improves mean performance for FFT, FWHT, and NTT, while deeper grouping regresses FFT and NTT. Their shared dependencies therefore expose a common trade-off with different preferred choices.

Logical intensity is not measured traffic: grouping also changes exchange, synchronization, and ownership. Matched profiles show increased register use and reduced active warps with deeper grouping. Section VI reports counters and timing ranges without assigning every change to one mechanism. The construction must expose these choices and constraints so that unfavorable allocations can be rejected for each family and workload.

C. Why a hierarchy and a common construction

The observations call for a **joint characterization of resource demand**. Spatial expansion needs concurrent execution, live state, and communication. Temporal reuse retains or moves values for later work at the chosen storage scope; it need not imply repeated external storage accesses.

Stage and data positions provide two allocation axes. Distinct resources can serve successive stages or independent regions; the same resources can instead be reused over either dimension. Their combination forms **hybrid dataflow**. Characterizing the resulting state, traffic, and parallel work makes the trade-off evaluable under a target's constraints. Stagewise execution, a spatial stage pipeline, and a whole-transform local execution are endpoints of this space when their resource demands fit.

Larger transforms can outgrow local storage or bounded compute groups. **Hierarchy repeats the construction under those limits.** Additional bands or recursive refinement preserve locality and handoff rules without fitting the complete intermediate array on chip. Logical depth and materialized groups remain distinct from physical storage levels; refinement adds no hardware capacity.

More batch provides independent work to amortize startup and cover transfers. Realizing that slack still requires lookahead, finite buffers, and available resources. A successor needs its actual predecessor set, not merely a contiguous prefix of producer output. The method must therefore derive both locality and readiness from the graph.

Stage-bit dependencies separate **semantics, workload, and hardware** inputs: arithmetic supplies local functions and labels; length and batch instantiate work; hardware supplies resources and service rates. The next section derives legal local work, handoffs, and resource demands across bounded storage scopes. Block-level and resident subgraph execution realize them on GPUs.

III. Hierarchical Hybrid Dataflow

The method connects **dependency structure, resource demand, and hardware constraints**. An **operator contract**

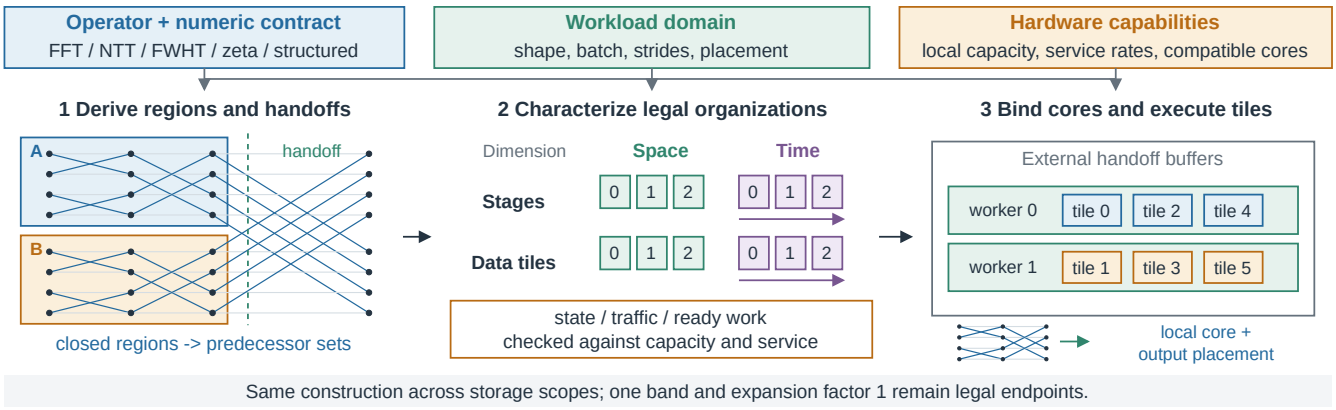


Fig. 3. CUBUTTERFLY maps labeled dependencies through space–time choices to bounded tile execution while retaining specialized arithmetic cores.

specifies arithmetic, coefficients, direction, normalization, precision, and logical input/output order; a **workload domain** specifies shape, batch, placement, strides, and storage layout; **hardware capabilities** specify resources and compatible cores. These inputs jointly determine performance. Figure 3 traces the construction: derive dependency-closed regions and handoffs, characterize spatial/temporal demands, then bind selected tile subgraphs to cores and storage scopes under target capacities and service rates. We develop it through a 16-point Hadamard example.

A. *Dependency-closed regions*

Local reuse requires a group containing all values needed by its operations. The stage-bit representation constructs such groups directly. **A stage band determines its local data domain.** For a radix-two transform, choose a consecutive stage band and let Q contain its pairing bits. Holding every other index bit fixed gives

$$\mathcal{I}(a, Q) = \{i \mid i \text{ agrees with } a \text{ outside } Q\}. \qquad (2)$$

Every pair update in the band changes only a bit in Q and thus stays inside this region. Assignments to the fixed bits partition the array into independent regions of $2^{|Q|}$ values. The band's computation on one region is a **dependency-closed region**. Closure is local to the band, whose inputs and outputs still connect to other bands.

Any ordered stage partition defines such regions, including one band spanning the complete transform when capacity and core support permit. Spatial expansion factors may also be one. Band width controls local reuse, while the number of regions assigned to a worker controls worker reuse. Arithmetic labels remain attached to the operations. In particular, an FFT band may require global-index-dependent twiddles, so a standalone smaller FFT is not automatically a compatible core.

A running example. For a 16-point unnormalized Hadamard transform with stage bits $(0, 1, 2, 3)$, write $i = 4r + c$. Two equal bands give four row subgraphs varying c, then four column subgraphs varying r. Each uses a four-point Hadamard core. Figure 2 places this two-band partition beside

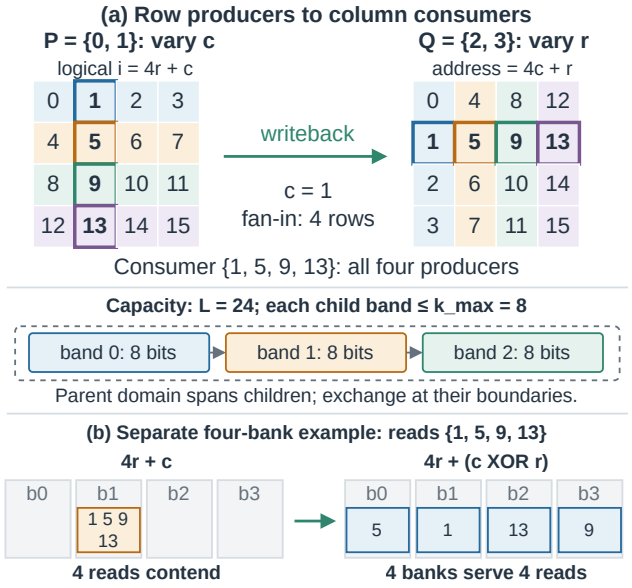


Fig. 4. The Hadamard example exposes dependency, capacity, and bank constraints. (a) Each column needs all row producers. (b) A separate four-bank swizzle distributes its reads; this map is not composed with (a). Counts are analytical.

stagewise and whole-transform partitions of the full graph. Region membership follows variable bits rather than original contiguity, allowing a representation chosen for physical stores and reads.

B. *Space–time organization and resource demand*

Closure defines independent work without fixing an execution group, launch, or pipeline depth. **Hierarchical hybrid dataflow assigns spatial resources and temporal reuse to stages and data.** Successive stages can reuse a worker or occupy different resources; independent regions can execute concurrently or stream through that worker. This organization repeats within regions and across storage scopes.

A worker can retain a small subgraph in local storage, compute groups can use an intermediate shared scope, and larger regions can use external storage handoffs. Refining a parent

TABLE I
SPATIAL AND TEMPORAL CHOICES IMPOSE DIFFERENT RESOURCE DEMANDS.

Choice	Demand to characterize
Stage in space	Concurrent cores, inter-stage transport, coefficients, and live pipeline state.
Stage in time	Retained state or feedback transfers, synchronization, and reuse of execution resources.
Data in space	Concurrent workers and state, input/output bandwidth, ports, and coefficient delivery.
Data in time	Sequential tile service, state lifetime, buffer capacity, and fill/drain overhead.

does not require its entire intermediate array to fit on chip. Hierarchical spatial and temporal mapping has precedents [19]. Here, butterfly closure and intersections determine the legal regions and handoffs.

Characterization separates demand from capability. At each scope, stage-space and data-space factors describe concurrent stage positions and independent pair work. Their temporal counterparts describe reuse of those resources. Table I distinguishes their demands at the selected storage and execution scope, with the chosen core and layout. These are logical organization choices; a worker count or compiler unrolling alone does not establish spatial execution.

For example, an ideal fully supplied stage pipeline with U_s stage positions and U_d pair units per position has $U_s U_d$ arithmetic units and consumes $2U_d$ input values per service step. At fixed arithmetic capacity, shifting resources toward stages reduces boundary supply per unit of work while increasing internal transport and pipeline depth. Actual cores, layouts, and resource sharing determine the achievable service rate.

At storage scope h, residency requires

$$\max_t \big(\mathrm{live}_h(t) + \mathrm{aux}_h(t)\big) \le C_h, \tag{3}$$

where live values, coefficients, and buffers are counted in bytes. Compiled register allocation and storage granularity further constrain the mapping. A tile subgraph that does not fit can be refined, rescheduled, or materialized at a larger scope. **Value residency differs from worker persistence.** A finite kernel may keep intermediates local, whereas a persistent worker may exchange them through external storage buffers.

Alongside peak live bytes, the characterization records boundary transfers, core work, and independent ready work. Capacity constrains feasibility; bandwidth, core service, and readiness constrain completion time. Feedback retained at an intermediate scope consumes its service rather than another external pass. Dependency analysis supplies exact logical sizes, intersections, and work; lowering and compilation determine physical ownership, allocation, and instructions; measurements supply service rates. Together they distinguish capacity exclusions from performance choices within the executable space. Logical traffic alone does not predict time.

From dependencies to an organization decision. A band spanning s distinct pairing bits has $BN/2^s$ independent regions. Retaining all values of one region at the chosen scope needs $e2^s$ bytes, before coefficients, buffering, and core scratch space. If G bands each read and write their values once, they transfer $2eBNG$ value bytes at that boundary. These counts connect a wider band to both greater retention and fewer independent regions. Actual transactions and cooperating threads refine these logical demands.

In the 16-point example, a four-value budget after reserving auxiliary storage permits at most two stages per region retained in local storage. Two bands minimize materialized passes under this capacity-only criterion, transferring 64 values instead of 128 for stagewise execution. A sixteen-value budget permits one band and 32 transfers, but exposes only one region per transform; performance still depends on its core mapping. At the four-value budget, a 32-point transform needs at least three such bands. The construction thus extends beyond two bands without increasing local storage. Feasibility, communication, and parallel organization jointly inform the physical choice.

The decision follows this order: dependencies determine legal bands and regions; capacity excludes mappings that do not fit; service rates and ready work rank the remaining retention choices. Producer–consumer intersections then determine handoff values and dependencies, while actual access groups and storage service constrain physical layout.

C. Communication and readiness

At a band boundary, changing pair partners connect closed regions through their index-set intersections. A producer and consumer with variable-bit sets P and Q share values when they agree on all bits fixed by both. Their shared variable bits $P \cap Q$ determine each nonempty intersection. Producers covering the remaining consumer bits $Q \setminus P$ form its predecessor set. This derives communication without enumerating scalar edges.

Fixing the bits outside $P \cup Q$ groups connected regions into a **boundary packet**. No edge crosses packets at that boundary. Within a packet, every producer and consumer intersect on the full pairing envelope. For complete-region publication, a consumer must therefore await every producer in its predecessor set. One-sided operators or finer publication units can expose finer dependencies.

In the example, a completed row cannot release a complete column. Independent transforms can nevertheless advance concurrently, and larger transforms may contain multiple packets within one transform. Packet membership is derived at each boundary, not assumed to form a fixed pipeline across the graph. A packet is a dependency domain, not a storage allocation or execution group. Its values can be distributed among workers and external storage buffers. This ownership description does not guarantee conflict-free service. Changing partners can concentrate successor reads on a few storage banks. Boundary placement must account for both producer and consumer access groups (Section IV-B).

D. Reuse across variants and dimensions

Reuse across stage orders, arithmetic, and tensor interpretations transports the labeled dependency relation, not another

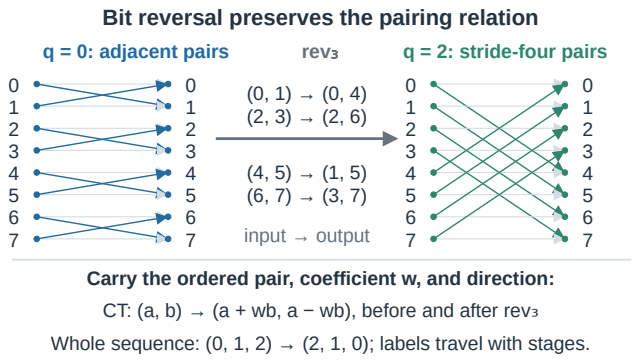


Fig. 5. Bit reversal maps adjacent pairs to strided pairs while preserving their arithmetic labels.

transform's numerical output. **Structural correspondence transports organizations.** Bit reversal maps an increasing-stride pairing envelope to a decreasing-stride one, preserving region and boundary relations. Physical layouts must still be evaluated for the transported access pattern.

Figure 5 shows this correspondence on eight indices. Reversing the three index bits sends adjacent pairs to stride-four pairs. Applying the same relabeling to every stage transports the complete envelope and its closed regions. It does not permit stages with different arithmetic labels to be exchanged arbitrarily.

Arithmetic labels move separately. A CT pair computes $(a+wb, a-wb)$ and a GS pair computes $(a+b, w(a-b))$. A GS pair with reciprocal twiddle and one-half scaling inverts the CT pair where those inverses exist. A full inverse also reverses stage order and applies the corresponding permutations. Hadamard changes the local labels to sums and differences, zeta/Möbius exchanges one-sided addition and subtraction, and structured stages require invertible local matrices for inversion. Numerical contracts remain explicit.

For separable power-of-two tensor axes, pairing bits can be labeled by axis and position. Axis-stage composition then uses the same closure and boundary rules on the tensor index domain. Axis order and inter-axis layout become mapping choices subject to the operator's factorization. This describes a shared construction, not a claim that every multidimensional or arithmetic variant already has an efficient lowering.

E. Correct composition

The arithmetic, placement, and scheduling rules must compose into the requested complete operator. A composed execution is correct when it covers each required operation once, each core implements its labels, every handoff preserves value identity, and every read occurs after production and before storage reuse. A topological induction suffices. Correct predecessor values reach the next operation through consistent layouts and valid lifetimes, and its arithmetic produces the required outputs. Thus local arithmetic, boundary layout, and scheduling carry distinct correctness obligations. The next section realizes these obligations with group-level and resident subgraph execution.

IV. Realizing the Method with Subgraph Execution

Lowering binds closed regions and boundary dependencies to GPU cores, storage, and synchronization, then generates kernels and a submission graph. A core declares logical value roles, worker ownership, and output visibility, not only input size. Local codelets, compute groups, generated kernels, and imported cores can share the construction with different resource costs. For scalar radix-2 FFT, FWHT, and NTT, common execution templates implement indexing, traversal, publication, and buffer exchange. Arithmetic traits supply pair operations, coefficients, and scaling. Specialized cores retain their numerical restrictions and may require different boundary realizations.

A. Successor-aligned writeback

A locally efficient producer may leave outputs inconveniently arranged for its consumer. Rather than add a permutation pass, it can incorporate the next placement into writeback when ownership and stores permit. For each value i, the producer's output and successor's input representations must agree,

$$\lambda_{\text{out}}(i) = \lambda_{\text{in}}(i). \tag{4}$$

Together with compatible ownership and publication, this defines **successor-aligned writeback**.

One construction moves the successor's variable stage bits into low-order address positions, placing its region in a contiguous range. In the Hadamard example, a producer can write logical index $4r + c$ at address $4c + r$, so the next band reads a column contiguously. Identity placement and explicit permutation remain alternatives. Consumer locality alone does not guarantee efficient stores, so the candidate cost includes both transactions and any staging.

B. Resolving boundary bank conflicts

An aligned layout can still serialize its consumers. Successor tiles gather different combinations than producers write. Distinct concurrent addresses can map to the same bank, whose limited ports serialize service even when values fit and successor reads are aligned.

We address this boundary mismatch with a **successor-aware bank mapping**. The producer and consumer use the same physical placement for each value. Candidate maps combine stage-bit placement with bijective address remapping, including XOR-based *swizzles*. Algorithm 1 gives a bank/offset construction for an aligned contiguous write group and a strided read group. It expresses the mapping in storage-bank coordinates, so the physical address encoding remains specific to the storage scope.

Algorithm 1. Map a producer–consumer boundary.

Given N words, $M = 2^k$ banks, stride $S = 2^s$,
$M \le S \le N$ and power-of-two N.
For each logical word $i \in [0, N)$
$b \leftarrow (i \oplus (i \gg s)) \,\&\, (M-1)$
$o \leftarrow i \gg k$
placement$[i] \leftarrow (b, o)$
Use this placement for producer stores and consumer loads.

For an aligned write group $i = aM + u$, $0 \le u < M$, the high-bit term is constant, so the bank is u XOR a constant. For a read group $i = c + vS$, $0 \le c < S$ and $0 \le v < M$, the bank is $(c \bmod M) \oplus v$. Both are permutations of the M banks for full groups; valid tails remain distinct. Within each offset, the low-bit mapping is also a permutation, giving a bijection over the storage domain. This establishes conflict freedom for these single-word groups on single-ported banks, rather than for arbitrary concurrent accesses.

For an illustrative square tile with power-of-two side P, P single-ported banks, and one word per value, consider

$$\lambda(r, c) = P\, r + (c \oplus r). \tag{5}$$

With bank index $\lambda \bmod P$, linear placement $Pr + c$ sends a whole column to one bank. XOR by a fixed row or column is a permutation, so the map above distributes either access across all P banks. In Figure 4b, consumer values $\{1, 5, 9, 13\}$ move from bank indices $(1, 1, 1, 1)$ to $(1, 0, 3, 2)$ without a separate data-reordering pass. On GPUs, such address remapping is commonly called a shared-memory bank swizzle [20].

Choose a common address map for the producer's write group and successor's read group. Conflict analysis uses actual transactions, value width, and bank capacity; vector values and broadcasts require separate checks. Outer-storage layouts instead require alignment, transaction-count, and coalescing checks. Thus layout and ownership are chosen jointly; one XOR formula need not suit every core or storage scope.

C. Residency, publication, and finite buffers

Correct placement still needs readiness and safe reuse. A resident tile subgraph tracks live values and unfinished consumers; an external handoff publishes a dependency unit. Successors start only after their predecessor sets are ready. A buffer slot is acquired before writing, published after production, and released after its final required read. More slots trade lookahead against live state and synchronization. Tail tiles retain their shape and release event.

Within-band pipelining and cross-band overlap are separate choices. Figure 6 shows cross-band readiness at the top: one completed Hadamard row leaves every column waiting for three others. After the full row band publishes, columns can overlap another transform's rows if resources permit; the buffer remains live through the final read. The bottom shows within-band load/compute/store overlap using two slots, which can operate despite a barrier before the next band. Cross-band overlap additionally needs ready predecessors and a free buffer. Batch supplies work for both, subject to lowering, sharing, and lifetime constraints.

D. Composing physical services

Placement and lifetime translate resource demand into completion time. A **service model** separates the startup, repeated work, and tail of a stage band's execution, avoiding the missing boundary costs of a sum of isolated core times,

$$T_{\text{group}} = T_{\text{start}} + R\, T_{\text{step}} + T_{\text{tail}}. \tag{6}$$

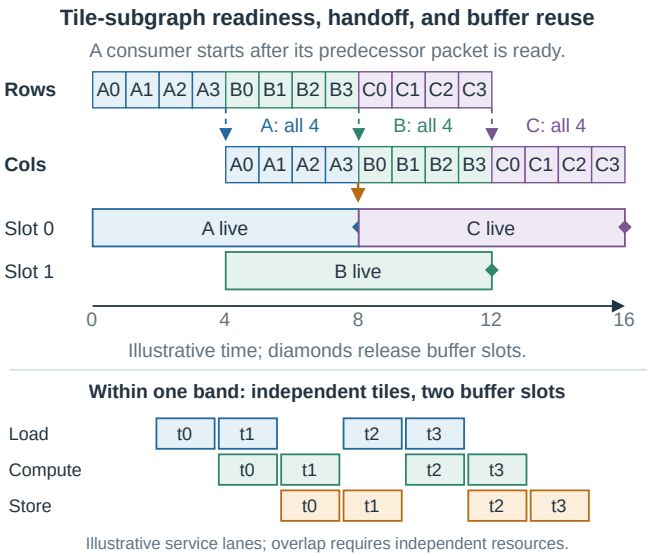


Fig. 6. Top: cross-band readiness for independent Hadamard transforms; C reuses slot 0 after A's final read. Bottom: within-band load/compute/store pipelining with two slots. Durations and lanes are illustrative.

The terms depend on the core, tile shape, layout, launch geometry, and storage scope. A complete-kernel measurement already includes its launch and tail. Bulk execution sums stage-band services plus boundary costs not already included.

For K tiles in a repeatable pipeline, the corresponding view is

$$T_{\text{pipe}} \approx T_{\text{fill}} + (K - 1)\,\text{interval} + T_{\text{drain}}. \tag{7}$$

Ready dependencies, finite-buffer reuse, and shared execution or memory resources limit the interval. Overlapped groups need not retain their isolated service rates. More independent work amortizes fill and drain and can cover transfer latency, but cannot exceed a saturated resource's throughput. The model ranks realizations under these assumptions rather than proving optimality.

V. Kernel Generation and Selection

The workload-dependent organization trade-off motivates selection even at fixed arithmetic. The inputs in Figure 3 feed the preparation and execution paths in Figure 7, instantiating subgraphs and handoffs. Storage constraints prune infeasible realizations; services and readiness rank the remainder. Residency, layout, and concurrency are chosen jointly, including handoff costs. Preparation produces a reusable plan whose complete device path is the unit of warm execution.

A. Generate compatible realizations

Generation combines stage bands and closed regions with compatible cores, storage, boundary layouts, and bulk or pipelined schedules. These coupled choices preserve the requested arithmetic, precision, direction, normalization, order, shape, and placement. Adapted plans include conversion costs. A legal organization without a compatible lowering remains a gap.

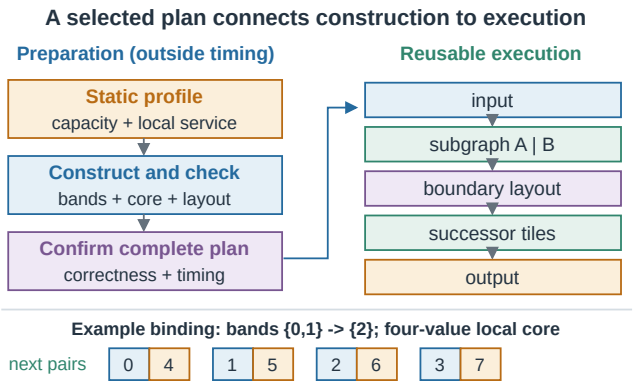


Fig. 7. Preparation constructs and checks subgraph plans, then the selected plan controls local execution and boundary exchange.

Static checks enforce dependencies and capacity; compilation exposes local allocation, intermediate-capacity, temporary-storage, and launch requirements. Numerical checks precede timing. Cache reuse requires matching semantic, workload, hardware, and compilation assumptions.

B. Estimate local services and their composition

To avoid remeasuring shared local work, services are keyed by arithmetic, core, tile shape, layout, and resource regime. Full and remainder tiles remain distinct when their work differs. Piecewise linear interpolation estimates nearby loads only within a compatible measured regime; missing coverage requires a local measurement or an explicit gap.

Composition follows the actual graph, admitting a group after predecessors and buffers are ready. Shared execution and memory capacity can lengthen concurrent service intervals. Submission costs are added only when absent from measured services, separating local work from contention and orchestration. These estimates guide preparation; the research contribution is the common organization characterization and its realization under hardware constraints. Search is the engineering mechanism that instantiates this space, not a claim of a new global optimization algorithm.

C. Bounded search and confirmation

Selection compares complete realizations under a finite measurement budget. Compatible cached plans and calibrated services avoid repeated preparation. Measurement modes use bounded enumeration or model-guided exploration; explicit evolutionary tuning adds multistart neighborhood mutation. Multidimensional selection measures complete candidates before refining axis choices. These strategies share semantic and resource constraints, while retaining the complete cost of each organization. Failed checks are excluded from timing and retained in the experiment record.

Complete-plan timing is the unit of confirmation, and static rankings remain distinguishable from measured selection. Organization comparisons distinguish explicit interventions from searched plans and retain their actual mappings, tuning conditions, and confirmation protocol. Preparation is separate from warm execution. The evaluation tests resulting performance and organization effects; finite search does not establish global optimality.

VI. Evaluation

A. Experimental setup

We compare operators, applications, fixed-core organization effects, and resource costs. Ratios are not pooled across unmatched studies.

Platforms and workloads. The main comparison uses A100 80GB PCIe (SM80), CUDA 12.4, and PyTorch 2.5.1. Its 66 requests cover FFT, NTT, FWHT, subset/superset zeta, and structured butterflies at ranks 1, 2, 3, 4, and 8. Precision, direction, normalization, order, shape, batch, placement, and strides define a request. The independent cyclic NTT study covers 90 requests, 32/64-bit words, multiple moduli, both directions, $N \in \{256, 65536, 1048576\}$ and $B \in \{1, 7, 16\}$. Low-precision studies comprise 22 fixed-mapping FP16/BF16 controls and nine public-selection FP16 requests over $N = 2^{12}$–2^{18}, $B = 4$–2048. BF16 public-library comparisons are deferred: cuFFT Xt's BF16 storage with mixed internal arithmetic differs from our native-BF16 scalar core.

Independent builds and replication. The resource, cross-core, and low-precision boundary controls use A100 40GB; the fixed-core FFT controls use 80GB, with an independent 40GB FP32 replication. An eleven-request 40GB FP16 supplement gives an installed-default cuFFT/cuButterfly GM of 0.858410× (0.738866–1.08757×), excluding it from the 80GB mean. Its omitted independent stage calibration leaves unseen-request ranking unvalidated. Cohorts preserve distinct device, build, and protocol identities; application timings do not evaluate the later FP16 build. All measurements are A100; Blackwell remains unmeasured.

Numerical and timing contracts. All 29 main-study integer and 90 cyclic NTT requests agree exactly. The 37 main-study floating requests have finite outputs and recorded errors, but no fixed acceptance threshold; their performance results remain exploratory. Complete-call latency includes required conversions. After ten warmups, all applicable operator baselines capture twenty calls per CUDA graph and report the median of seven event-timed replays divided by twenty. Replays within one process are not independent trials. Steady-state timing excludes search, compilation, setup, capture, and checking. Profiler timings do not replace ordinary timings; ranges are not confidence intervals, and near-parity differences do not establish significance. NTT controls check every batch; local-grouping and cross-core floating controls retain rounded maximum errors. Random weights and short-chain cryptography establish neither task quality nor production security. FFT-L scale controls repeat one seeded transform, scaled by 2^{-6}, across batch and backends; transforms are not independent error samples. Boundary controls use saved inputs and one process per endpoint. For 138 finite-field compositions, public-default and replayed cyclic mappings pass random and boundary inputs exactly in all 552 measurements.

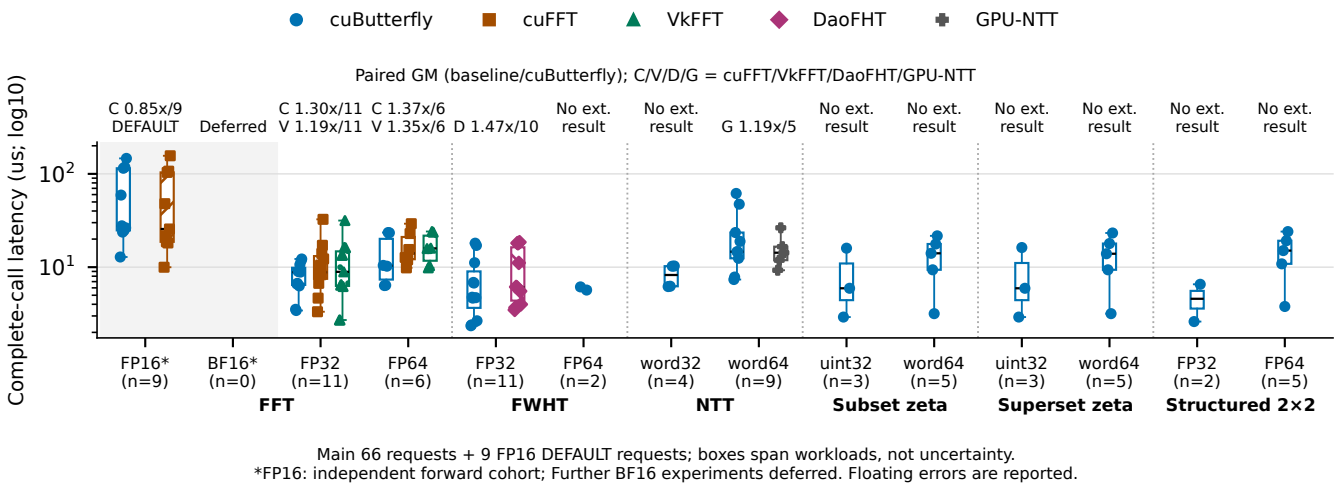


Fig. 8. Complete-call comparisons group operators by precision; FP16 FFT uses a separate public-selection cohort, while further BF16 experiments are deferred.

TABLE II
MATCHED OPERATOR SPEEDUPS RETAIN SEPARATE STUDY POPULATIONS.

Operator	Baseline	Pairs	GM	Faster	PRO
Main 66-request study					
FFT FP32	cuFFT	11	1.298×	9/11	–
FFT FP32	VkFFT	11	1.190×	5/11	–
FFT FP64	cuFFT	6	1.366×	5/6	–
FFT FP64	VkFFT	6	1.354×	6/6	–
FWHT FP32	DaoFHT	10	1.474×	9/10	–
NTT w64	GPU-NTT	5	1.188×	4/5	–
Independent FP16 public DEFAULT study					
FFT FP16	cuFFT	9	0.854×	1/9	–
FFT BF16	cuFFT	0	–	–	–
Independent cyclic NTT study (90 requests)					
NTT w32	GPU-NTT	18	1.400×	18/18	–
NTT w64	GPU-NTT	36	1.265×	33/36	–

B. Performance comparisons

Question 1: External operator comparisons. Figure 8 shows the 66 main requests and nine independent FP16 public selections. Boxes span workloads; annotations give paired geometric means (GM) and counts. Table II separates matched populations by competitor and precision. Ratios are baseline/CUBUTTERFLY latency; faster counts ratios above one. Missing pairs remain unmeasured. FP64 FFT and FP32 FWHT win most pairs; FP32 FFT retains small-length and multidimensional losses. Five word64 NTT pairs yield 1.188× on average. Three-trial replay gives 1.175×; forward $N = 65536, B = 1$ remains near parity.

Native computation and adaptation. Eight directly supported cuFFT comparisons give 1.102×; seven including normalization/layout adaptation give 1.385×. Two rank-4/8 axis-composition replays retain 1.925× and 2.748× against cuFFT compositions. FP64 FWHT, zeta, and structured butterflies lack matched GPU baselines. The separate 34-request CPU study uses two processes on a shared host and a different timing boundary.

Finite-field range and batch. The cyclic study obtains 54 exact GPU-NTT pairs, with a geometric mean of 1.308× and 51 observed ratios above one. The 18 word32 pairs achieve 1.400× and the 36 word64 pairs 1.265× (Table II). The comparator's modulus limits (2^{30} for word32, 2^{61} for word64) exclude 36 requests, which retain exact CUBUTTERFLY results without external ratios. Three word64, $B = 1$ requests yield 0.972–0.999×, retaining the small-batch losses. Eighteen pairs were measured together; 36 reuse the unchanged CUBUTTERFLY timings with separately collected baseline timings on the same GPU and protocol.

Where aggregate gains do not carry over. Three-process confirmations give 0.838× against cuFFT for FP64 FFT at $N = 2^{20}, B = 16$ and 0.939× against DaoFHT for FP32 FWHT at $N = 2^{15}, B = 32$. The nine-request FP16 cohort yields 0.854× against cuFFT, with ratios of 0.733–1.067×. The $N = 2^{18}, B = 32$ request takes 146.53 μs versus cuFFT's 156.31 μs; the other eight requests remain slower. Default and bounded measured selection choose the same mapping in all nine requests: confirming the tested candidates retains these losses. cuFFT matches storage precision and semantics but does not expose its internal arithmetic.

Question 2: End-to-end integration. Applications share inputs, precision, keys, and weights as applicable. Complete Graph calls include surrounding GPU work, packing, conversions, and output materialization; reusable setup stays outside timing [21].

Figure 9 separates 46 current Poisson/filter, CKKS, and bootstrap comparisons from historical and independent replays. Hollow markers identify single-process filters; CKKS diamonds show 12-request batch GMs.

For FP32 periodic Poisson, the three (N, B) pairs are $(1024, 4)$, $(2048, 4)$, and $(2048, 16)$. Complete-call medians are 0.200, 0.893, and 3.245 ms for CUBUTTERFLY, giving 1.082–1.107× over Torch/cuFFT and 1.097–1.125× over CuPy. Twenty event samples include input refresh, both transforms, frequency multiplication, and independent output

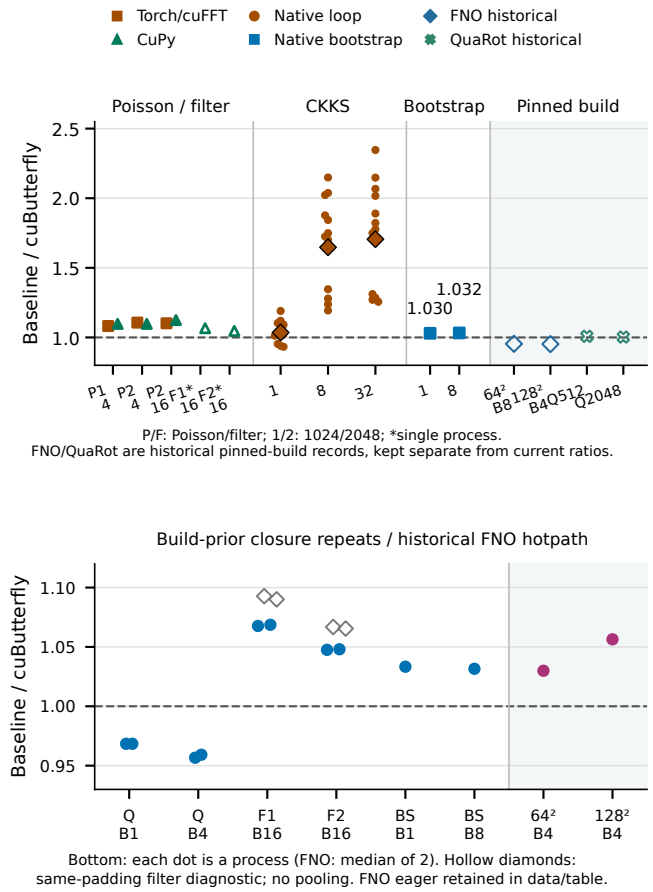


Fig. 9. Complete-call speedups retain the main application and historical cohorts above, with independent application replays below. Cohorts are not pooled.

copies. CPU, changed/restored-input, and lifetime checks pass; relative L_2 error is below 5.7×10^{-7}.

Fixed-filter endpoints use $1024^2/2048^2$ images, $B = 16$, a shared 31^2 filter, cached filter spectra, and identical $1080^2/2100^2$ padding. Complete-call medians are 1.409/5.647 ms versus CuPy's 1.502/5.912 ms, giving 1.066/1.047× from one process per endpoint. An earlier recomputed-filter contract won three of nine requests; filter caching changes the application contract.

The CKKS matrix crosses four operations—key switching, square/relinearize/ rescale, square/rescale/key switching, and rotation—with $B = 1, 8, 32$. It uses key-switch methods I/II at $N = 16384$ and method I at $N = 65536$. Loop/stage-batch selection precedes twenty holdout rounds of ten Graph replays.

All 36 selected paths produce elementwise-identical ciphertexts. Their geometric mean is 1.428× over native loop, with batch-group means of 1.036, 1.648, and 1.705× at $B = 1, 8, 32$. The matched-batch control below separates integration from transform gains. Small-parameter regular bootstrap at $N = 4096$, $B = 1/8$ yields 1.030/1.032× over HEonGPU, using five event samples per backend. Both CKKS cohorts use `sec_level=none`.

Historical pinned-build FNO at $64^2/128^2$, $B = 8/4$, gives 0.954/0.952×; random-weight QuaRot at prefill/decode

TABLE III
INDEPENDENT GRAPH COMPARISONS SHOW PROCESS RANGES, OR THE MEDIAN OF PROCESS MEDIANS FOR FNO. EAGER AND SAME-PADDING DIAGNOSTICS REMAIN SEPARATE.

Scenario	Type	Baseline	Ratio
QuaRot, B1	FWHT	QuaRot/Dao	0.968
QuaRot, B4	FWHT	QuaRot/Dao	0.957–0.959
Filter 1024^2, B16	FFT	CuPy	1.068–1.069
Filter 2048^2, B16	FFT	CuPy	1.048
Bootstrap, B1	NTT	HEonGPU	1.033
Bootstrap, B8	NTT	HEonGPU	1.032
FNO 64^2, B4	FFT	Torch	1.030
FNO 128^2, B4	FFT	Torch	1.056

TABLE IV
CKKS GAINS REMAIN WITH MATCHED BATCH INTEGRATION.

B	Requests	External Loop	Integration controls Batch	Faster native
1	12	1.036	1.290	1.036
8	12	1.648	1.183	1.182
32	12	1.705	1.182	1.182

512/32 and 2048/128 gives 1.008/1.003× [22], [23]. These complete-call records include input refresh and output materialization and remain outside current-batch aggregates. QuaRot makes no trained-model quality claim.

Independent application cohorts (Table III) retain four random-weight QuaRot losses at prefill/decode 128/8, $B = 1/4$ (0.957–0.968×), including baseline-first and treatment-first orders. Their FP16 adapter composes FP32 conversion, FWHT, scaling, and narrowing; these adaptation costs are not quantified separately. FNO at $64^2/128^2$, $B = 4$, gives 1.030/1.056× in Graph mode, including input copy, full forward/replay, output clone, and synchronization. Its eager ratios are 0.955/0.957×, reported separately. These cohorts neither replace historical records nor enter the 46-comparison aggregate.

Matched application integration. The CKKS control gives native NTT cores the same gather/scatter and Graph organization as our stage-batch path, without assuming an upstream batch API. Table IV compares native loop, matched batch, and the faster native alternative selected per request.

Across the same 36 requests, the mean is 1.218× over matched native batch and 1.131× over the faster native alternative, with 33 of 36 ratios above one. The three losses occur at $B = 1$; both larger batch groups retain about 1.182× over the faster native alternative. The smaller gains quantify the integration effect.

C. Ablations

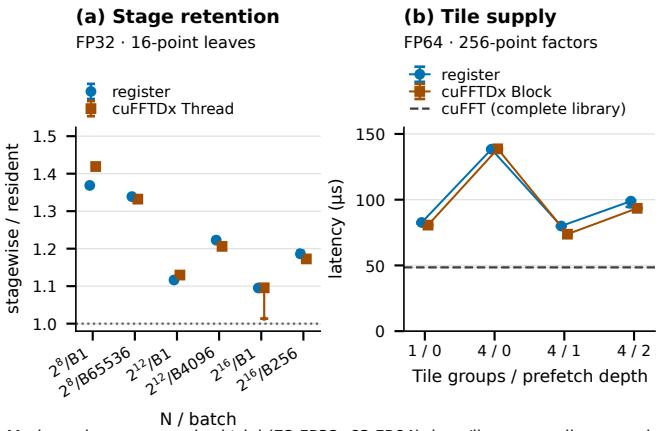
Question 3: What does organization contribute with a fixed core? Figure 10a crosses native register and cuFFTDx Thread 16-point leaves with stagewise exchange and a resident two-stage prefix, sharing later stages. This separates fixed-core organization changes from core substitution at fixed organization.

Six FP32 workloads across four cells and three trials produce 72 checked rows with 10,000 warmups and 100 repetitions. Residency improves all six workloads for both cores, with geometric means of 1.217× and 1.221× (Figure 10a). Core substitution at fixed organization contributes 1.042× and 1.046×. Organization gains therefore exceed the core-substitution effect. Organization changes ownership, storage, and launches: at $N = 256, B = 1$, the alternatives launch 16 and 256 threads. Independent replication gives 1.226× and 1.231×.

A stronger organization reference tests the size of the gain. The FP64 control at $N = 65536, B = 16$ fixes two 256-point factors, four local transforms/CTA, and 16 elements/thread. Register and cuFFTDx Block cores share a factor template and writer-aligned handoff; only tile groups/CTA and prefetch depth vary (Figure 10b).

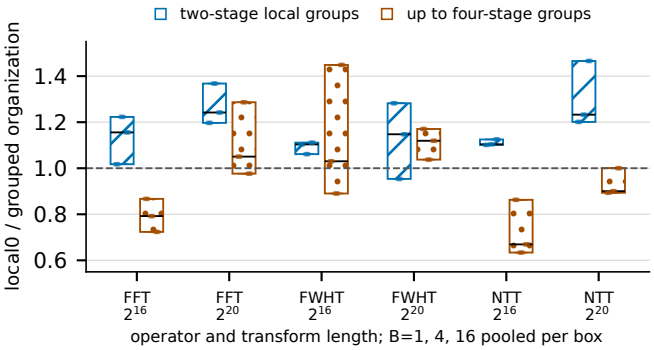
At four tile groups per CTA, one-step prefetch reduces native-core latency from 138.377 to 79.974 μs and cuFFTDx latency from 138.684 to 73.762 μs, giving 1.730× and 1.880×. Against the faster one-tile synchronous alternative (82.671/80.657 μs), gains shrink to 1.034/1.093×. Depth two regresses for both cores. cuFFT takes 48.503 μs and remains faster on this request. Relative L2 error against CPU FP64 is below 7.4×10^{-16}. The stronger organization reference reduces the gain; imported leaves do not constitute complete-library baselines.

The same organization principle across arithmetic families. The cross-operator control fixes scalar radix-2 FFT/FWHT FP64 and word64 NTT, varying local stage grouping. It covers 18 workloads ($N = 2^{16}, 2^{20}$ and $B = 1, 4, 16$), three organizations, and three independent trials, for 162 observations. Two-stage grouping gives GM speedups over stagewise execution of 1.196×, 1.106×, and 1.199× for FFT, FWHT, and NTT; up to four-stage groups give 0.932×, 1.103×, and 0.816×, respectively. Deeper grouping helps only FWHT on average.



Markers show every paired trial (72 FP32, 63 FP64); bars/lines are cell summaries.
Panels use independent FFT workloads; not a cross-precision comparison.

Fig. 10. Independent fixed-core controls: (a) FP32 stagewise/resident ratios; (b) FP64 tile-grouping and prefetch latencies. Markers expose all 72 FP32 trial ratios and 63 FP64 timing rounds; both panels vary organization under two cores, while cuFFT in (b) is a complete-library reference.



18 workloads × 3 organizations × 3 trials = 162 observations;
boxes span batches and markers show every trial. A100-40GB M1 cohort.

Fig. 11. Stage grouping across 18 workloads. Boxes pool the three batches and their three trials for each operator and length; points show all 162 observations.

Transfer to finite-field compositions. Across 138 coset, negacyclic, and RNS requests, replaying the existing selected cyclic mapping per modulus yields a 1.691× geometric mean over the internal public-default, with request ratios of 1.051–2.996×. Each request geometrically averages two input-pattern ratios. Core and organization may both change, so this is neither fixed-core attribution nor joint RNS search. Default runs first; seven Graph replays are not independent process trials.

Mechanism choices and their interactions. Fixed-core toggles isolate mechanism costs; equal-budget re-search with a mechanism disabled tests its contribution to selection.

For the FFT factor-streamed control at $N = 256$, $B = 16384$, enabling slice overlap changes native-core latency from 127.560 to 132.346 μs and imported-core latency from 126.331 to 130.957 μs. The imported core's advantage is only about 1%.

Disabling stage overlap, resident fusion, or successor layout and repeating eight-candidate search tests selection sensitivity for FFT ($N = 16384, B = 16$), FWHT and NTT ($N = 16384, B = 4$). FFT and FWHT timing differences are small; NTT restricted-search regret is 0.7% for resident fusion and 2.9% for successor layout, with overlap within measurement noise. Two illegal unrestricted mappings leave six accepted candidates versus eight restricted candidates, limiting attribution.

In separate layout/depth controls, writer-aligned layout loses in its three tested pairs, while greater resident depth wins in three pairs. Pipeline batching loses to bulk in 27/27 comparisons: at $N = 4096$, $B = 64$, ordinary timings are 29.266 μs bulk and 45.636 μs pipeline. Overlap must offset fragmentation.

These searches compare finite candidate sets, not global optima.

Crossings that do not improve performance. A separate cohort crosses two cores and organizations for FWHT FP32 at $N = 2^{16}$ and word64 NTT at $N = 2^{20}$, both at $B = 4, 16$. FWHT increases suffix columns per CTA

from one to four, separately fixing radix-2 and radix-4 local algorithms. NTT changes resident batch traversal from one transform to two, separately fixing shared radix-4 and warp-register radix-2 cores. Throughput falls to 0.872–0.937× and 0.797–0.911×, respectively. Wide three-trial ranges provide no evidence of positive transfer or latency hiding; these are controlled organizations, not optimized plans.

D. Analysis

Question 4: Which resource costs accompany organization choices? Nsight Compute (NCU) counters and Nsight Systems (NSYS) timelines characterize internal controls, not causes of external-library gains. Ordinary timings determine performance.

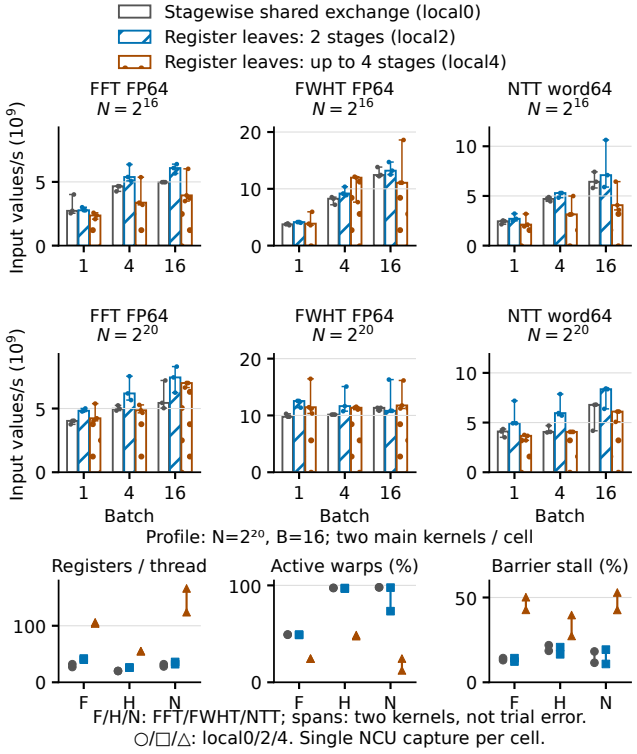


Fig. 12. Local organization changes throughput and resource pressure. Top: medians, trial points, and ranges. Bottom: two main-kernel records per organization at $N = 2^{20}, B = 16$; spans are not trial uncertainty.

Scaling depends on organization, not only on available work. Figure 12 varies local shared exchange under fixed scalar radix-2 arithmetic and two bands: 8+8 stages at $N = 2^{16}$, 10+10 at $N = 2^{20}$. The reference exchanges every stage. All 54 configurations retain three process trials (162 observations). Throughput counts complex FFT or scalar FWHT/NTT inputs, not common FLOPs. Two-stage leaves improve all families on average; deeper grouping loses for FFT and NTT. It changes exchange, synchronization, and ownership within fixed outer boundaries while exposing fewer owners.

Local grouping and resource pressure. Figure 12 bottom matches $N = 2^{20}, B = 16$: two kernels each launch 16,384 CTAs of 256 threads. Two-stage leaves change measured DRAM traffic little. Four-stage leaves increase register demand and reduce active-warps fractions from about 49% to 24% for FFT, 97% to 48% for FWHT, and 98% to 12–25% for NTT. Their barrier-stall percentages also rise. Fewer exchange boundaries need not reduce waiting; shared-access counts and instruction-level attribution remain unmeasured.

Counter boundaries. Kernel-only NCU totals omit the Dao FWHT diagnostic's 2 MiB copy; underfilled stagewise controls are not tuned baselines.

Stage grouping and launches. In the six radix-2 controls, matched captures contain 12 versus 2 transform kernels at $N = 2^{12}$ and 18 versus 3 at $N = 2^{18}$. Grouping removes launches around the same core. Capture traffic includes setup and replay, not ordinary complete-call cost.

Overlap and launch structure. In the factor-streamed FFT control, both overlap variants launch three kernels and measured L2 traffic differs by less than 0.2%. The imported core uses fewer registers, yet its complete call improves by only about 1%. In the bulk/pipeline control, NSYS observes overlap but eight launches per pipeline call versus two for bulk. Observable overlap alone does not imply a gain.

Cross-core mapping traces. At $B = 16$, the NTT shared-core treatment replaces a specialized schedule with a generic schedule around the same leaf. Registers rise from 48 to 52, while measured active warps remain near 24% and DRAM traffic falls. The warp-register core uses 239 registers and about 12.5% active warps in both organizations; its treatment increases captured DRAM traffic. These changes do not isolate one cause of the timing loss.

Boundary traffic and remaining local costs. A matched FP16 capture at $N = 4096, B = 256$ retains both stage bands and their packed arithmetic instruction counts. Revised ownership and writeback reduce store sectors per request from 32 to 4 in both bands, while total warp instructions fall from 34.32 to 11.48 million. Barrier-stall fractions fall from 75.2/55.8% to 10.5/14.0%. Boundary service improves, but these counters do not imply eightfold fewer external-memory bytes. Some shared-memory bank conflicts increase. The complete two-band call improves from 126.26 to 43.16 μs, but its 34.20 μs single-band alternative and 11.11 μs cuFFT call remain faster.

FP16 organization preferences also change with load. At FP16 $N = 4096$, radix-4 single-band/two-band times are 13.11/9.27 μs for $B = 4$ and 33.59/39.83 μs for $B = 256$; at $N = 8192, B = 32$ they are 32.36/19.35 μs. The large-batch two-band points use grouped tiles. Retaining single-band execution and unit expansion therefore matters. Missing single-band implementations above $N = 8192$ in this control do not establish hardware infeasibility.

The later FP16 cohort imports paired local transforms into the same factor template. Public selection retains one local-core pass at $N = 4096/8192$ and two factors at larger lengths, with writer-aligned intermediate layouts. At larger batches it selects four tile groups per block and one-step prefetch. Factor kernels use 96–128 registers/thread and roughly 15–17 active warps, versus cuFFT's 32–48 registers and 35–59

TABLE V
TRANSFORM SYSTEMS EXPOSE DIFFERENT EXECUTION AND PLACEMENT CHOICES.

System	Ops.	Work unit	Stage split	Data split	Placement
FFTW [2]	FFT, DCT/ DST	codelets	recursive plans	vector loops	buffered copies
SPIRAL / FFTX [12], [13]	FFT apps.	formulas / plans	algebraic rewrite	NR	generated glue
VkFFT [9], [24]	FFT, DCT	GPU kernels	Four-Step	grouped FFTs	reshuffled layout
GPU-NTT [4], [5]	NTT	CUDA kernels	Merge / Four-Step	NR	global-memory exchange
Flash FFTConv [3]	FFT conv.	CUDA kernels	matrix factors	batch / hidden	retain / materialize
HadaCore [25]	FWHT	CTA vectors	16-point groups	warp chunks	SMEM transpose
CU BUTTERFLY	FFT, NTT, FWHT	tile subgraphs	bit bands	tile groups	successor-aligned stores

Representative mechanisms. Apps./conv.: applications/convolution; CTA: threadblock; SMEM: shared memory. NR: the partition is unresolved in this comparison, not unsupported. Matrix factors and 16-point groups describe local transforms.

warps. Register and shared-memory capacity jointly restrict residency without isolating one cause. At $N = 2^{18}, B = 32$, two factor passes nevertheless outperform cuFFT's three-pass implementation. Organization gains coexist with a local-core efficiency gap.

VII. RELATED WORK

Related systems optimize transform formulas, arithmetic cores, tensor programs, and hardware mappings.

Transform generation and specialized execution. FFTW combines codelets with empirical planning; SPIRAL/FFTX generates transforms and surrounding computation from algebraic descriptions [2], [12], [13]. VkFFT generates GPU implementations with grouped strided transforms and layout choices. GPU-NTT optimizes locality, kernel count, block size, and shape [4], [5], [9], [24]. FlashFFTConv combines tensor-core FFT decomposition, batch/hidden tiling, and memory-aware fusion [3]. HadaCore maps FWHT chunks to warps and vectors to threadblocks with shared-memory transposes [25]. These systems optimize both arithmetic and organization.

Table V separates their representative operators, units, stage decompositions, independent-work mappings, and boundary mechanisms. A GPU kernel is a launched program; a tile subgraph groups computation and values for scheduling and residency. Kernels can execute tile subgraphs, and larger subgraphs can span kernels. Stage fusion can retain intermediates while reusing execution resources over time, so it does not alone establish a spatial pipeline.

Tensor compilation and tile scheduling. TVM separates computation from scheduling [26]; Ansor searches hierarchical schedule sketches [27]; ROLLER constructs hardware-aligned tiles [28]. TensorIR provides block signatures, outer transformations, and tensorization [29]. Welder connects producer/consumer tiles and schedules reuse through traffic and footprint estimates [30]. These establish tiling and reusable compute interfaces. Their generality alone does not verify transform-specific support. For butterflies, the changing pair relation identifies the values needed by a stage group and the producers that a successor must await, before choosing a physical tile or arithmetic core. This separates dependency construction from a particular kernel implementation while preserving the operator's arithmetic, direction, ordering, and precision contract.

Hierarchical dataflow and positioning. Eyeriss analyzes data movement under controlled resources; Timeloop separates workload, architecture, and hierarchical spatial/temporal mappings [19], [31]. Applying these principles to butterflies requires legal resident tile subgraphs and their remaining communication. GPU blocks also share resources, precluding an assumption of dedicated pipeline hardware.

Our distinction concerns the common dependency interface. SPIRAL/FFTX derives implementations from algebraic transform decompositions, including surrounding computation [12], [13]. Timeloop evaluates hierarchical mappings from workload and architecture descriptions [19]. These precedents motivate separating arithmetic from organization; that separation alone is not our novelty. We derive the butterfly-specific link between dependencies and resource demand: stage-bit sets construct dependency-closed regions, and their intersections derive communication and readiness (Section III). Spatial and temporal organization then determines the demands evaluated against hardware constraints. The contribution is this butterfly-specific construction, not the general idea of spatial/temporal dataflow or a claim that prior systems cannot express equivalent programs. It retains specialized arithmetic across workloads and hardware constraints. The fixed-core experiments in Section VI-C isolate this organization effect from arithmetic substitution.

VIII. CONCLUSION

CUBUTTERFLY derives local regions, communication and readiness relations, and resource demands from butterfly dependencies. Hierarchical hybrid organization reuses this construction across arithmetic cores and workloads while evaluating its demands under target hardware constraints. Fixed-core experiments isolate organization gains; library and application comparisons expose both benefits and losses. The evidence supports a reusable organization method on the tested A100 workloads and numerical contracts, rather than a universal mapping or hardware optimum.

REFERENCES

[1] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.

[2] M. Frigo and S. G. Johnson, "The design and implementation of FFTW3," *Proceedings of the IEEE*, vol. 93, no. 2, pp. 216–231, 2005. [Online]. Available: https://www.fftw.org/fftw-paper-ieee.pdf

[3] D. Y. Fu, H. Kumbong, E. Nguyen, and C. Ré, "FlashFFTConv: Efficient convolutions for long sequences with tensor cores," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://arxiv.org/abs/2311.05908

[4] A. Ş. Özcan and E. Savaş, "Two algorithms for fast GPU implementation of NTT," Cryptology ePrint Archive, Paper 2023/1410, 2023. [Online]. Available: https://eprint.iacr.org/2023/1410

[5] A. Ozcan, A. Javeed, and E. Savas, "High-performance number theoretic transform on GPU through radix2-CT and 4-step algorithms," *IEEE Access*, vol. 13, pp. 87 862–87 883, 2025. [Online]. Available: https://doi.org/10.1109/ACCESS.2025.3570024

[6] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. De Sa, "QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 48 630–48 656. [Online]. Available: https://proceedings.mlr.press/v235/tseng24a.html

[7] T. Dao, A. Gu, M. Eichhorn, A. Rudra, and C. Ré, "Learning fast algorithms for linear transforms using butterfly factorizations," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97, 2019, pp. 1517–1527. [Online]. Available: https://proceedings.mlr.press/v97/dao19a.html

[8] NVIDIA, "cuFFT library documentation," https://docs.nvidia.com/cuda/cufft/index.html, accessed September 16, 2026; experimental binary versions are recorded separately.

[9] VkFFT Project, "VkFFT: GPU fast Fourier transform library," https://github.com/DTolm/VkFFT, official implementation and documentation; accessed September 16, 2026.

[10] GPU-NTT Project, "GPU-NTT: Merge and 4-step number theoretic transforms on GPUs," https://github.com/Alisah-Ozcan/GPU-NTT, official implementation; accessed September 16, 2026. Measured adapter versions and semantic scope are recorded separately.

[11] Dao AI Lab, "Fast Hadamard transform in CUDA, with a PyTorch interface," https://github.com/Dao-AILab/fast-hadamard-transform, official implementation; accessed September 16, 2026.

[12] M. Püschel, J. M. F. Moura, J. R. Johnson, D. Padua, M. M. Veloso, B. W. Singer, J. Xiong, F. Franchetti, A. Gačić, Y. Voronenko, K. Chen, R. W. Johnson, and N. Rizzolo, "SPIRAL: Code generation for DSP transforms," *Proceedings of the IEEE*, vol. 93, no. 2, pp. 232–275, 2005.

[13] F. Franchetti, D. G. Spampinato, A. Kulkarni, D. T. Popovici, T. M. Low, M. Franusich, A. Canning, P. McCorquodale, B. V. Straalen, and P. Colella, "FFTX and SpectralPack: A first look," in *IEEE 25th International Conference on High Performance Computing Workshops (HiPCW)*, 2018, pp. 18–27. [Online]. Available: https://www.spiral.net/doc/papers/hipc_2018.pdf

[14] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[15] W. M. Gentleman and G. Sande, "Fast Fourier transforms," in *AFIPS Fall Joint Computer Conference*, 1966.

[16] T. G. Stockham, Jr., "High-speed convolution and correlation," in *AFIPS Spring Joint Computer Conference*, 1966.

[17] NVIDIA, "NVIDIA Ampere GPU Architecture Tuning Guide," https://docs.nvidia.com/cuda/ampere-tuning-guide/index.html, accessed September 16, 2026.

[18] ——, "cuFFTDx: Execution methods," https://docs.nvidia.com/cuda/cufftdx/api/methods.html, accessed September 16, 2026.

[19] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, "Timeloop: A systematic approach to DNN accelerator evaluation," in *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2019, pp. 304–315.

[20] NVIDIA, "CUDA Programming Guide: Shared-memory bank swizzling," https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/async-copies.html\#shared-memory-bank-swizzling, accessed October 29, 2026.

[21] HEonGPU Contributors, "HEonGPU: Gpu-accelerated fully homomorphic encryption library," https://github.com/Alisah-Ozcan/HEonGPU, open-source implementation; accessed September 21, 2026. Application source revision is pinned separately in the experiment manifest.

[22] NeuralOperator Contributors, "NeuralOperator: Learning in infinite dimensions," https://github.com/neuraloperator/neuraloperator, open-source implementation; accessed September 21, 2026. Application source revision is pinned separately in the experiment manifest.

[23] QuaRot Contributors, "QuaRot: Outlier-free 4-bit inference in rotated LLMs," https://github.com/spcl/QuaRot, open-source implementation; accessed September 21, 2026. Application source revision is pinned separately in the experiment manifest.

[24] D. Tolmachev, "VkFFT—a performant, cross-platform and open-source GPU FFT library," *IEEE Access*, vol. 11, pp. 12 039–12 058, 2023. [Online]. Available: https://doi.org/10.1109/ACCESS.2023.3242240

[25] K. Agarwal, R. Astra, A. Hoque, M. Srivatsa, R. Ganti, L. Wright, and S. Chen, "HadaCore: Tensor core accelerated Hadamard transform kernel," arXiv:2412.08832, 2024. [Online]. Available: https://arxiv.org/abs/2412.08832

[26] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An automated end-to-end optimizing compiler for deep learning," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 578–594. [Online]. Available: https://www.usenix.org/conference/osdi18/presentation/chen

[27] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen, J. E. Gonzalez, and I. Stoica, "Ansor: Generating high-performance tensor programs for deep learning," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 863–879. [Online]. Available: https://www.usenix.org/conference/osdi20/presentation/zheng

[28] H. Zhu, R. Wu, Y. Diao, S. Ke, H. Li, C. Zhang, J. Xue, L. Ma, Y. Xia, W. Cui, F. Yang, M. Yang, L. Zhou, A. Cidon, and G. Pekhimenko, "ROLLER: Fast and efficient tensor compilation for deep learning," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 233–248. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/zhu

[29] S. Feng, B. Hou, H. Jin, W. Lin, J. Shao, R. Lai, Z. Ye, L. Zheng, C. H. Yu, Y. Yu, and T. Chen, "TensorIR: An abstraction for automatic tensorized program optimization," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. ACM, 2023, pp. 804–817. [Online]. Available: https://arxiv.org/abs/2207.04296

[30] Y. Shi, Z. Yang, J. Xue, L. Ma, Y. Xia, Z. Miao, Y. Guo, F. Yang, and L. Zhou, "Welder: Scheduling deep learning memory access via Tile-graph," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, 2023, pp. 701–718. [Online]. Available: https://www.usenix.org/conference/osdi23/presentation/shi

[31] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *Proceedings of the 43rd Annual International Symposium on Computer Architecture*, 2016, pp. 367–379. [Online]. Available: https://eems.mit.edu/wp-content/uploads/2016/04/eyeriss_isca_2016.pdf